

COMP1870 Theoretical Foundations of Computer Science I

School of Computer Science, University of Leeds

Worksheet - Week 1.1

This worksheet contains a combination of *formative activities* (which contribute towards your learning) and *summative activities* (which you will complete and submit to be assessed as part of your portfolio).

- Exercises marked with red borders are summative and must be submitted in Gradescope.
- Other exercises may be covered in tutorial sessions.
- Exercises marked by (*) are advanced, and may take some time to complete.
- Exercises marked by (E) are exam-style questions. You should expect to see questions like this in the exam.

Expectations:

- 1. Timeliness** You should complete all of the activities in the order provided and submit your portfolio evidence on PebblePad before the deadline.
- 2. Presentation:** You should present all of your work clearly and concisely following any additional guidance provided by the module staff.
- 3. Integrity:** You are responsible for ensuring that the evidence you submit as part of your portfolio is **entirely your own work**. You can find out more about Academic integrity on the Skill@library website. All work you submit for assessment is subject to the academic integrity policy.

Feedback: Feedback on formative activities will be provided via Lab classes and tutorials. Feedback on evidence submitted as part of the portfolio will be available on Gradescope.

Support opportunities: Support with the activity sheet is available in tutorials and the Math Lunch Hour. Individual support is available via office hours.

Expected completion date: Friday 9 October 2025

All portfolio tasks are rated AMBER by the university: AI tools can be used in an assistive role. You are permitted to use AI tools for specific defined processes within the assessment. Within this assessment you may use Generative AI to:

- help you start to understand more complex ideas by providing accessible summaries.
- test your knowledge against a piece of content.
- help identify and correct spelling mistakes and grammatical errors in your work.
- provide feedback and advice on your overall coding style.

You must not use Gen AI to:

- produce written content (code, comments or any other content) which you then submit as your work, regardless of whether you make changes to it.
- rewrite or make changes to any of your work.

Learning outcomes

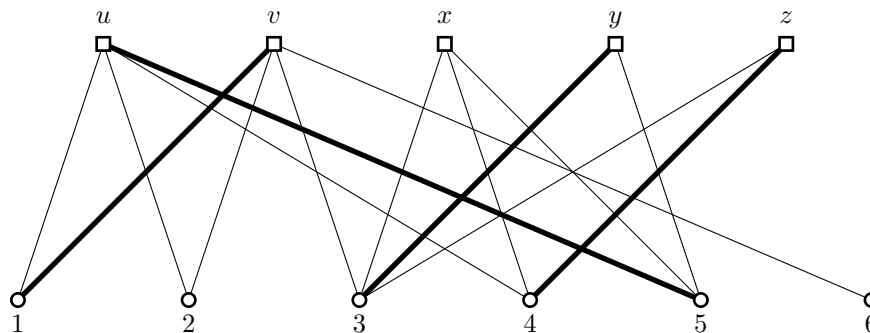
On completion of this activity sheet you will have:

1. demonstrated an understanding of maximum-matching problem,
2. defined augmenting paths in bipartite graphs,
3. constructed a truth table for a propositional formula,
4. reasoned about logical expressions using logical equivalencies,
5. translated logical statements in natural language to expressions in propositional logic.

Tutorials on Wednesday and Thursday will cover material from Week 1, Lecture 1. This material is in exercises 1–4. Tutorials on Monday and Tuesday will cover material from Week 1, Lecture 2. This material is in exercises 6–12.

Questions

1. Let G be the graph shown below.

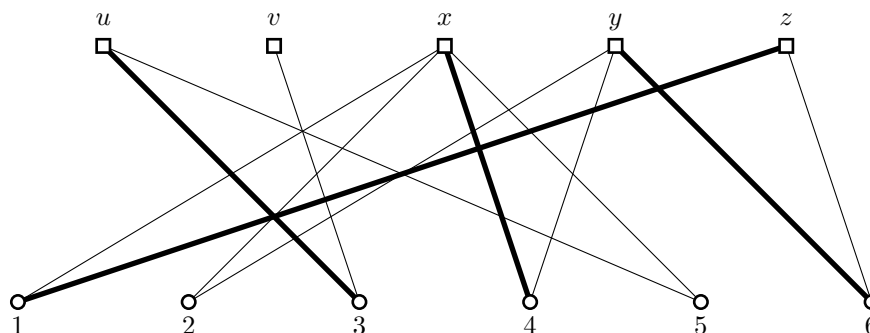


Let X be the set of vertices marked by circles, and let Y be the set of vertices marked by squares. Let M be the matching consisting of thicker edges.

Apply Algorithm 1 from the lecture notes to this input to find an augmenting path. Start by setting L to be the list $[2, 6]$ and marking every vertex as unvisited except for the vertices in L . Every time we go through the **while** loop between lines 5 and 31, we change the list L . Write down L each time you reach the end of this loop. When you process the unvisited vertices in line 9, process them in alphabetical or numerical order. What is the augmenting path produced as output by the algorithm?

2. Use the augmenting path you discovered in Question 1 to produce a matching that is larger by one edge. What is the matching you produce?
3. Let M be a matching in a bipartite graph. Review the definition of an augmenting path. Let P be an augmenting path. Prove that if we remove from M the edges that are in P and add the edges of P that are not in M , then we obtain a matching that contains one more edge than M .
4. The augmenting paths algorithm finds a maximum-sized matching. A matching is *maximal* if it is not possible to add another edge to it and still have a matching. Give an example of a matching that is maximal but not maximum-sized. Give an example of a graph where every maximal matching is also maximum-sized.

5. Let G be the graph shown below.



Let X be the set of vertices marked by circles, and let Y be the set of vertices marked by squares. Let M be the matching consisting of thicker edges.

Apply Algorithm 1 from the lecture notes to this input to find an augmenting path. Start by setting L to be the list $[2, 5]$ and marking every vertex as unvisited except for the vertices in L . Every time we go through the **while** loop between lines 5 and 31, we change the list L . Write down L each time you reach the end of this loop. When you process the unvisited vertices in line 9, process them in alphabetical or numerical order. What is the augmenting path produced as output by the algorithm?

6. Complete truth tables for the following statements.

- (a) $P_1 \Rightarrow (P_1 \wedge (\neg P_2))$
- (b) $(P_1 \vee (\neg P_2)) \Rightarrow (P_2 \wedge P_3)$
- (c) $(P_1 \wedge \neg(P_1)) \Rightarrow P_2$
- (d) $(P_1 \wedge (P_2 \vee P_3)) \Leftrightarrow (P_1 \vee (P_2 \wedge P_3))$

7. (E) If P_1 is true, P_2 is false, and P_3 is true, what is the truth value of

$$((P_1 \vee P_2) \wedge P_3) \Rightarrow (P_1 \wedge P_3) \quad ?$$

8. Use the logical equivalences in Section 2.1 of the lecture notes to show that

$$\neg(P_1 \Rightarrow (P_2 \vee (\neg P_3))) \text{ is equivalent to } (P_1 \wedge (\neg P_2)) \wedge P_3.$$

At each step, state which equivalence you are using.

9. Use the logical equivalences in Section 2.1 of the lecture notes to show that

$$(P_1 \Rightarrow P_2) \vee P_3 \text{ is equivalent to } (P_1 \wedge (\neg P_2)) \Rightarrow P_3$$

At each step, state which equivalence you are using.

10. Use the logical equivalences in Section 2.1 of the lecture notes to show that

$$\neg((P_1 \wedge P_2) \vee (P_2 \Rightarrow P_3)) \text{ is equivalent to } (((\neg P_1) \vee (\neg P_2)) \wedge P_2) \wedge (\neg P_3)$$

At each step, state which equivalence you are using.

11. (*) Let G be a bipartite graph where the vertex set is divided into two sets, X and Y , and every edge joins an edge in X to an edge in Y . Let M be a matching of G .

Give a proof that every augmenting path joins a vertex in X to a vertex in Y .

12. (*) Let M be a matching in a complete bipartite graph. Let M' be a matching that contains at least one more edge than M . Prove that there is an augmenting path for M .
(This will be proved later in the module.)